ENLACE PELUQUERÍA

Documento de Traspaso — Estado del Proyecto y Plan de Continuación

Este documento resume todo lo avanzado, lo que falta y los prompts exactos para continuar con Trae.

Bajo Deslinde SPA | John Andrés Arancibia Maureira | La Serena, Chile | 2025

CONTEXTO PARA TRAE:

Estamos construyendo Enlace Peluquería — app mobile-first de gestión y reservas para peluquerías.

La app es parte del ecosistema Enlace: red social + apps verticales para microemprendedores chilenos.

El proyecto tiene un backend Node.js/Express con Prisma/SQLite (apps/api) y un frontend React/Vite (apps/web).

Este documento es el punto de partida para continuar sin necesitar el historial del chat anterior.

01 Estado Actual del Proyecto

Lo que ya está hecho y funciona

Backend — apps/api/src/ (COMPLETADO)

Archivo	Estado	Qué hace
index.js	LISTO	Todas las rutas Express: auth, shop, services, clients, appointments, gallery, enlace, public booking
availability.js	LISTO	Calcula slots disponibles respetando horario, pausas y citas existentes
enlace.js	LISTO	Outbox para sincronización con la red social Enlace, se ejecuta cada 30s
auth.js	LISTO	Login con cookie-session y bcrypt
prisma/schema.prisma	LISTO	Modelos: Shop, Client (con colorFormula), Service, Appointment, GalleryItem, EnlaceOutbox
prisma/dev.db	LISTO	Base de datos SQLite con migraciones aplicadas incluyendo add_color_formula

Frontend — apps/web/src/ (EN PROGRESO)

Archivo	Estado	Notas
index.css	REEMPLAZADO	Paleta Enlace: fondo #dce8f6, cards #fff, accent #2563eb, naranja #f5a020
App.jsx	REEMPLAZADO	Rutas: /b/:slug (booking), /admin (shell con 8 subpáginas)
admin/AdminShell.jsx	REEMPLAZADO	Header gradiente azul + ENLA(azul)CE(naranja) + toggle Enlace + bottom nav 5 tabs
admin/Dashboard.jsx	PENDIENTE	Mostrar citas de hoy, KPIs, próxima cita, acciones rápidas — llamar a /api/appointments?date=hoy y /api/shop
admin/Agenda.jsx	PENDIENTE	Calendario semanal táctil + lista citas del día + sheet de detalle — llamar a /api/appointments?date=X
admin/Clientes.jsx	PENDIENTE	CRM: lista, ficha, historial, colorFormula — llamar a /api/clients
admin/Caja.jsx	PENDIENTE	Citas completadas del día + extras — llamar a /api/appointments?date=X
admin/Services.jsx	PENDIENTE	Catálogo mobile-first — llamar a /api/services
admin/Enlace.jsx	PENDIENTE	Toggle disponibilidad, vincular, publicar foto/oferta — llamar a /api/enlace/*
admin/Gallery.jsx	PENDIENTE	Galería mobile-first — llamar a /api/gallery
pages/Booking.jsx	PENDIENTE	Flujo 4 pasos mobile: servicio→fecha→hora→datos — llamar a /public/shops/:slug/*
pages/BookingConfirm.jsx	PENDIENTE	Confirmación con botón WhatsApp — llamar a /public/shops/:slug/appointments/:id
pages/BookingCancel.jsx	PENDIENTE	Cancelación self-service con verificación de teléfono

02 Referencia Completa del API Backend

Todos los endpoints disponibles para el frontend

Rutas públicas — sin autenticación (para el booking del cliente)

Método	Endpoint	Para qué
GET	/public/shops/:slug	Info del negocio (nombre, horario, política)
GET	/public/shops/:slug/services	Lista de servicios activos
GET	/public/shops/:slug/availability?date=YYYY-MM-DD&serviceId;=X	Slots horarios disponibles
POST	/public/shops/:slug/appointments	Crear reserva (invitado)
GET	/public/shops/:slug/appointments/:id	Ver detalle de una cita
POST	/public/shops/:slug/appointments/:id/cancel	Cancelar cita (requiere phone en body)

Rutas admin — requieren sesión activa (cookie)

Método	Endpoint	Para qué
POST	/api/auth/login	Login { email, password }
POST	/api/auth/logout	Cerrar sesión
GET	/api/me	Verificar sesión activa
GET	/api/shop	Info del shop + entitlements del plan
PATCH	/api/shop	Actualizar configuración del negocio
GET	/api/services	Lista de servicios
POST	/api/services	Crear servicio
PATCH	/api/services/:id	Editar servicio
GET	/api/clients	Lista de clientes con historial
POST	/api/clients	Crear cliente
PATCH	/api/clients/:id	Editar cliente (incluye colorFormula)
DELETE	/api/clients/:id	Eliminar cliente
GET	/api/appointments?date=YYYY-MM-DD	Citas de un día (incluye service + client)
POST	/api/appointments	Admin crea cita/walk-in
PATCH	/api/appointments/:id/status	Cambiar estado { status }
GET	/api/gallery	Lista de fotos del portafolio
POST	/api/gallery	Subir foto { description, category, photos: [urls] }
DELETE	/api/gallery/:id	Eliminar foto
GET	/api/enlace/status	Estado de conexión Enlace + cola pendiente

POST	/api/enlace/link	Vincular { enlaceProfileId, code }
POST	/api/enlace/unlink	Desvincular
POST	/api/enlace/toggle	Toggle disponibilidad { disponible, msg }
POST	/api/enlace/sync-catalog	Sincronizar catálogo con Enlace

IMPORTANTE: El frontend usa fetch con credentials: 'include' (cookie de sesión).

Funciones disponibles en apps/web/src/lib/api.js:

apiGet(url) — GET con credenciales

apiPost(url, body) — POST con JSON body

apiPatch(url, body) — PATCH con JSON body

apiDelete(url) — DELETE con credenciales

Todas lanzan Error si res.ok es false. Siempre usar try/catch o .catch().

03 Sistema de Diseño — Paleta y Componentes

Cómo debe verse cada página para ser consistente con Enlace

Variables CSS disponibles (definidas en index.css)

Variable	Valor	Usar para
--bg	#dce8f6	Fondo general de la app
--card	#ffffff	Cards, panels, bottom nav
--border	#c5d5e8	Bordes de cards y separadores
--hover	#eaf2fb	Fondo de inputs y estados hover
--text	#1a2447	Texto principal
--muted	#5a7a9a	Texto secundario, labels
--accent	#2563eb	Color principal Enlace (azul)
--orange	#f5a020	Color acento Enlace (naranja)
--green	#10b981	Éxito, completado, disponible
--red	#ef4444	Error, cancelado, peligro
--safe-b	env(safe-area-inset-bottom)	Padding bottom nav en iPhone
--safe-t	env(safe-area-inset-top)	Padding header en iPhone

Reglas de diseño que toda página debe respetar

1. Mobile-first — todo debe verse perfecto en pantalla de 390px de ancho.
2. Sin sidebar — la navegación es solo el bottom nav del AdminShell.
3. Scroll vertical — cada página tiene su propio scroll interno, nunca el body.
4. Cards blancos — background: var(--card), border: 0.5px solid var(--border), border-radius: 12-14px.
5. Header azul — solo el AdminShell lo tiene, las páginas internas no tienen header propio.
6. Botones grandes — mínimo 44px de alto para ser táctiles.
7. ENLA en azul #2563eb y CE en naranja #f5a020 — siempre así en todos los textos del logo.
8. WhatsApp verde — usar #25D366 para todos los botones de WhatsApp.
9. Estados de cita: CONFIRMED=verde, PENDING=rojo, IN_PROGRESS=naranja, COMPLETED=verde, CANCELED=gris.
10. Usar CSS inline en JSX — sin archivos CSS separados por componente.

04 Prompts Exactos para Trae — Páginas Pendientes

Copiar y pegar en Trae en este orden exacto

INSTRUCCIÓN GENERAL PARA TRAE ANTES DE CADA PROMPT:

Al inicio de cada prompt, incluir esta frase:

'Crea el archivo [nombre].jsx en apps/web/src/pages/admin/ con el siguiente comportamiento.'

Trae debe generar el archivo completo con CSS inline, sin importar hojas de estilo externas,

usando las funciones apiGet/apiPost/apiPatch del archivo ../../lib/api.js

PROMPT 7 — Dashboard.jsx

```
/*** PROMPT 7 — copiar completo en Trae ***/

Crea el archivo Dashboard.jsx en apps/web/src/pages/admin/

Este es el Inicio del panel de administración. Usa useOutletContext() para obtener { shop }.

Al montar, llama a estos endpoints:

  GET /api/appointments?date=HOY  (fecha en formato YYYY-MM-DD)

Muestra:

1. Saludo: 'Hola [shop.name]!' + fecha de hoy en español (ej: Lunes 4 de Mayo)

2. Fila de 4 KPIs en cards: Citas hoy / Online (tipo online) / Completadas / Ingresos (suma precio
 completadas)

3. Card 'Próxima cita': la primera cita con status PENDING o CONFIRMED ordenada por hora.

   Muestra: hora, nombre del cliente (guestName o client.fullName), servicio, duración.

   Botón WhatsApp verde (#25D366) si tiene teléfono.

4. Lista de todas las citas de hoy ordenadas por hora con: hora, nombre, servicio, badge de estado
.

5. Dos botones al fondo: '+ Nueva cita' (navega a /admin/agenda) y '■ Walk-in' (navega a /admin/a
genda?walkin=1)

Paleta: fondo #dce8f6, cards blancos, accent #2563eb, estados: CONFIRMED=verde, PENDING=rojo, IN_P
ROGRESS=naranja.

CSS completamente inline. Mobile-first. Sin librerías externas.

Manejar estado loading con un texto 'Cargando...' mientras llega la data.
```

PROMPT 8 — Agenda.jsx

```
/*** PROMPT 8 — copiar completo en Trae ***/

Crea el archivo Agenda.jsx en apps/web/src/pages/admin/
Es la vista de agenda con calendario semanal y lista de citas del día.

Estado inicial: fecha seleccionada = hoy.

Al montar y cada vez que cambia la fecha, llama a:
  GET /api/appointments?date=YYYY-MM-DD
  GET /api/services (para el formulario de nueva cita)
  GET /api/clients  (para el formulario de nueva cita)

Estructura de la pantalla:
1. SELECTOR DE SEMANA (sticky arriba):
   - Fila con los 7 días de la semana actual (Lun a Dom).
   - Cada día muestra: nombre corto (Lun), número, punto azul si tiene citas.
   - El día seleccionado tiene fondo azul #2563eb con texto blanco.
   - Botón '+Nueva' a la derecha que abre el formulario de nueva cita.

2. LISTA DE CITAS DEL DÍA:
   - Cada cita es un card con: hora inicio, hora fin, nombre cliente, servicio, badge estado, prec
io.
   - Al tocar una cita abre un panel inferior (position absolute abajo) con:
     * Detalle completo de la cita
     * Botón WhatsApp si tiene teléfono
     * Botón 'Rebooking' que crea la misma cita +14 días llamando a POST /api/appointments
     * Botones para cambiar estado: llamar a PATCH /api/appointments/:id/status
     * Botón eliminar

3. FORMULARIO NUEVA CITA (panel inferior al presionar +Nueva):
   - Select de servicio (cargado de /api/services)
   - Select de cliente existente O campos para cliente nuevo (nombre + teléfono)
   - Input de fecha (por defecto la fecha seleccionada)
   - Input de hora
   - Input de precio (se pre-llena con el precio del servicio)
   - Textarea de notas
   - Botón 'Confirmar cita' → POST /api/appointments
   - Botón 'Walk-in' → mismo POST /api/appointments con tipo: 'walkin'

Paleta y reglas iguales al Dashboard. CSS inline. Mobile-first.
```

PROMPT 9 — Clientes.jsx

```
/*** PROMPT 9 — copiar completo en Trae ***/

Crea el archivo Clientes.jsx en apps/web/src/pages/admin/
Es el CRM de clientes. Al montar llama a GET /api/clients.

Tres vistas (estado interno: 'lista' | 'ficha' | 'editar'):

VISTA LISTA:
  - Buscador de texto (filtra por fullName y phone en el frontend, sin llamada extra al API).
  - Botón '+Nuevo' que abre la vista editar con campos vacíos.
  - Cada cliente: avatar inicial, nombre, teléfono, badge categoría (VIP/Regular/Nuevo),
    número de visitas (count de appointments), indicador naranja si tiene colorFormula guardada.
  - Al tocar un cliente navega a vista ficha.

VISTA FICHA:
  - Avatar grande con inicial, nombre, teléfono, badge categoría.
  - 3 KPIs: Visitas / Total gastado / Última visita.
  - Si tiene notas: mostrar en card gris.
  - Si tiene colorFormula: mostrar en card naranja con emoji ■.
  - Botón WhatsApp verde si tiene teléfono.
  - Botón 'Editar' que navega a vista editar.
  - Historial de citas: lista con servicio, fecha, hora, precio, badge estado.

VISTA EDITAR (crear o editar):
  - Campos: fullName*, phone, email, notas.
  - Selector de categoría: Regular / VIP / Nuevo / Inactivo (botones táctiles).
  - Campo especial con borde naranja para colorFormula: 'Fórmula de color'
    placeholder: 'Ej: Tinte 7.3 + oxidante 20vol, 40min'
  - Botón 'Guardar' → POST /api/clients (nuevo) o PATCH /api/clients/:id (editar).
  - Botón 'Eliminar' solo en modo editar → DELETE /api/clients/:id.

CSS inline. Mobile-first. Sin librerías externas.
```

PROMPT 10 — Caja.jsx

```
/*** PROMPT 10 — copiar completo en Trae ***/

Crea el archivo Caja.jsx en apps/web/src/pages/admin/
Es el registro de ingresos y cierre diario.

Al montar y al cambiar la fecha, llama a:
  GET /api/appointments?date=YYYY-MM-DD

Muestra:
1. Input tipo date para seleccionar el día (por defecto hoy).
2. Grid de 4 KPIs:
   - Servicios: suma de precio de citas con status COMPLETED
   - Extras: suma de ingresos extras (guardados en localStorage como 'caja_extras')
   - Egresos: suma de egresos (localStorage)
   - Total: servicios + extras - egresos
3. Lista de citas completadas del día: nombre cliente, servicio, hora, precio.
4. Sección 'Extras / Egresos' con botón '+Agregar':
   - Formulario: concepto (texto), monto (número), tipo (ingreso/egreso).
   - Los extras se guardan en localStorage con clave 'caja_extras' como array de objetos
     { id, fecha, concepto, monto, tipo }.
   - Lista de extras del día con botón eliminar.
5. Card de resumen con el total del día en naranja (#f5a020) y número de servicios.

Nota: los extras en localStorage son temporales — son para registros rápidos del día.
En una versión futura se migrarán a la base de datos.
CSS inline. Mobile-first.
```

05 Prompts para las Páginas Públicas de Booking

La experiencia del cliente al reservar desde su celular

PROMPT 11 — Booking.jsx (flujo del cliente)

```
/*** PROMPT 11 — copiar completo en Trae ***/
Crea el archivo Booking.jsx en apps/web/src/pages/
Es la página pública de reservas que ve el CLIENTE en su celular.
Usa useParams() para obtener el slug.

Al montar llama a:
  GET /public/shops/:slug          (info del negocio)
  GET /public/shops/:slug/services (servicios activos)

Flujo de 4 pasos con barra de progreso visual:

PASO 1 — Elegir servicio:
  Muestra servicios agrupados por categoría con nombre, descripción, duración y precio.
  Al seleccionar un servicio avanza al paso 2.

PASO 2 — Elegir fecha:
  Grid de 16 fechas disponibles (próximos 28 días, solo días activeDays del shop).
  Muestra día de la semana, número y mes.
  Al seleccionar fecha avanza al paso 3.

PASO 3 — Elegir hora:
  Al entrar llama a: GET /public/shops/:slug/availability?date=X&serviceId;=Y
  Muestra los slots disponibles en grid de 3 columnas.
  Si no hay slots: mensaje 'Sin horarios disponibles' + botón 'Otro día'.
  Al seleccionar hora avanza al paso 4.

PASO 4 — Datos del cliente:
  Card resumen del servicio seleccionado (nombre, fecha, hora, precio).
  El cliente elige: 'Continuar como invitado' o 'Crear cuenta Enlace'.
  Campos: nombre completo*, WhatsApp*, email (solo si crea cuenta), notas.
  Política de cancelación del shop.
  Botón 'Confirmar reserva' → POST /public/shops/:slug/appointments
  Body: { serviceId, date, time, name, phone, email?, notes? }
  Si tiene éxito: navega a /b/:slug/confirm/:appointmentId

Header: gradiente azul con logo ENLA(azul)CE(naranja) + nombre del shop.
CSS inline. Mobile-first. Sin librerías externas.
```

PROMPT 12 — BookingConfirm.jsx

```
/*** PROMPT 12 — copiar completo en Trae ***/
Crea el archivo BookingConfirm.jsx en apps/web/src/pages/
Usa useParams() para obtener slug e id.
Al montar llama a: GET /public/shops/:slug/appointments/:id
Muestra:
1. Emoji de celebración ■ grande.
2. Título '¡Reserva confirmada!'
3. Card con detalles: servicio, fecha, hora, duración, total.
4. Botón WhatsApp verde (#25D366) que abre:
   wa.me/[shop.phone]?text=Hola [shop.name]! Confirmé mi cita para [servicio] el [fecha] a las [hora].
5. Link 'Cancelar esta reserva' → navega a /b/:slug/cancel/:id
6. Link '← Hacer otra reserva' → navega a /b/:slug
CSS inline. Mobile-first.
```

PROMPT 13 — BookingCancel.jsx

```
/*** PROMPT 13 — copiar completo en Trae ***/
Crea el archivo BookingCancel.jsx en apps/web/src/pages/
Usa useParams() para obtener slug e id.
Al montar llama a: GET /public/shops/:slug/appointments/:id
Muestra el detalle de la cita y pide verificación del teléfono:
1. Card con: servicio, fecha, hora, estado actual.
2. Si ya está cancelada/completada: mostrar mensaje y no permitir cancelar.
3. Si está activa: campo para ingresar WhatsApp para verificar identidad.
4. Botón 'Cancelar mi reserva' → POST /public/shops/:slug/appointments/:id/cancel
   Body: { phone: valorIngresado }
   Si error PHONE_MISMATCH: mostrar 'El teléfono no coincide con la reserva'.
   Si éxito: mostrar '■ Reserva cancelada correctamente' + link para reservar de nuevo.
CSS inline. Mobile-first.
```

06 Cómo Verificar que Todo Funciona

Checklist de pruebas antes de lanzar

#	Prueba	Cómo hacerla	Esperado
1	API corriendo	curl http://localhost:5174/health	{ ok: true }
2	Frontend carga	Abrir http://localhost:5173	Redirige a /admin/login
3	Login funciona	Email y contraseña del seed	Redirige a /admin con header azul
4	Bottom nav visible	Ver la pantalla de admin	5 tabs abajo: Inicio Agenda Clientes Caja Más
5	Dashboard carga	Ir a Inicio	Muestra citas de hoy o 'Sin citas'
6	Agenda carga	Ir a Agenda	Calendario semanal + lista de citas
7	Clientes carga	Ir a Clientes	Lista de clientes del seed
8	Booking público	Abrir /b/[slug]	Flujo 4 pasos en mobile
9	Nueva cita funciona	Desde Agenda + Nueva	Cita aparece en la lista
10	Toggle Enlace	Header azul, botón Online	Llama a /api/enlace/toggle sin error 500

Credenciales del seed (revisar apps/api/prisma/seed.js para confirmar)

El seed crea un Shop y un AdminUser de prueba. Busca en seed.js el email y password para hacer login.

SI HAY ERRORES 401 O REDIRIGE A LOGIN EN CADA REQUEST:

El proxy de Vite debe estar configurado para redirigir /api/* y /public/* al backend.

Revisar apps/web/vite.config.js — debe tener:

server: { proxy: { '/api': 'http://localhost:5174', '/public': 'http://localhost:5174' } }

Si no está configurado así, Trae debe agregarlo.

07 Prompt de Arranque para la Próxima Sesión con Claude

Copiar y pegar al inicio de la nueva sesión para dar contexto completo

PEGAR ESTE TEXTO AL INICIO DE LA NUEVA SESIÓN DE CLAUDE:

```
/*** Copiar todo esto al inicio de la nueva sesión con Claude ***/
Continúo construyendo Enlace Peluquería — app mobile-first de gestión y reservas.
Es parte del ecosistema Enlace: red social + apps verticales para microemprendedores chilenos.

STACK:
- Backend: Node.js/Express + Prisma + SQLite en apps/api/ (puerto 5174)
- Frontend: React/Vite en apps/web/ (puerto 5173+)
- Repo: D:\Nueva carpeta\peluqueria\peluqueria (monorepo npm workspaces)

LO QUE ESTÁ LISTO:
- Todo el backend (index.js con 20+ rutas, auth, enlace outbox, availability)
- Schema Prisma con colorFormula en Client, migraciones aplicadas
- Frontend: index.css (paleta Enlace), App.jsx (rutas), AdminShell.jsx (bottom nav)

LO QUE FALTA (frontend):
- Dashboard.jsx, Agenda.jsx, Clientes.jsx, Caja.jsx (páginas admin)
- Services.jsx, Enlace.jsx, Gallery.jsx (páginas admin — reemplazar con UX mobile)
- Booking.jsx, BookingConfirm.jsx, BookingCancel.jsx (páginas públicas)

PALETA: fondo #dce8f6, cards #fff, accent #2563eb, naranja #f5a020.
ENLA siempre azul #2563eb y CE siempre naranja #f5a020.
CSS inline en JSX. Sin librerías de UI externas. Mobile-first.

Necesito el código completo de [NOMBRE DEL ARCHIVO] conectado al API real.
```